

Loop Engineering: 에이전트를 프롬프트하는 시스템 설계를 위한 Anthropic 플레이북

공개 PDF에서 재구성한 한국어 번역본
원본 TeX가 아닌, 공개 PDF 기반 재구성·번역본

Abstract—지난 2년 동안 모델 출시 속도에 맞춰 여러 “XX Engineering” 용어가 등장했다. 이 글은 2026년 6월 Peter Steinberger, Boris Cherny, Addy Osmani가 거의 동시에 언급했고, Osmani가 글로 정리하며 이름 붙인 새로운 용어 Loop Engineering을 다룬다. Prompt, Context, Harness Engineering이 사람이 에이전트에게 더 잘 지시하는 방법을 다루었다면, Loop Engineering은 사람이 직접 지시하는 자리에서 한 걸음 물러나 그 지시를 자동으로 수행하는 시스템을 설계하는 일이다. 이 번역본은 원문의 정의, 내층 stack, 한 turn을 이루는 다섯 move, generator와 evaluator의 분리, 실제 사례, 비용, 첫 loop를 만드는 절차를 한국어로 옮긴 것이다.

색인어

에이전트 AI, software engineering, autonomous agent, coding agent, generator – evaluator, scheduling, automation.

I. 서론: Loop Engineering이 실제로 뜻하는 것

Prompt Engineering 강좌는 여전히 팔리고, Context Engineering이라는 말도 막 자리 잡았으며, Harness Engineering 역시 이제 막 정리되었다. 그런데 이제 Loop Engineering까지 등장했다. 모델 release가 빨라질 때마다 새로운 “XX Engineering”이 붙는 것처럼 보여 냉소하고 싶어지는 것도 자연스럽다. 하지만 이 용어는 조금 다르다. 이것은 사람이 일을 더 잘하게 만드는 기술이라기보다, 사람이 일을 직접 수행하는 자리에서 물러나도록 만드는 설계다. 이전 용어들은 모두 사람이 키보드 앞에 앉아 에이전트에게 한 줄씩 지시한다고 가정했다. Loop Engineering은 그 가정을 지운다. 사람은 더 이상 loop 안쪽의 작업자가 아니라, loop 바깥에서 loop를 설계하는 사람이 된다.

I-A 한 줄 정의

이 용어를 글로 정리해 이름 붙인 Addy Osmani의 정의는 짧다. Loop Engineering은 “에이전트를 프롬프트하던 자기 자신을, 그 일을 대신 수행하는 system으로 바꾸는 것”이다. 이제 사람은 에이전트에게 입력을 한 줄씩 건네지 않는다. 대신 그런 입력이 자동으로 전달되도록 system을 설계한다. 핵심은 “자기 자신을 대체한다”는 대목이다. 이는 engine

역할을 하던 자리에서, engine을 설계하는 자리로 이동하는 일이다.

I-B 한 주 안에 세 사람이 불을 붙였다

2026년 6월의 한 주 동안 여러 실무자가 거의 같은 결론에 도달했다. OpenClaw의 저자 Peter Steinberger는 이제 코딩 에이전트를 직접 프롬프트할 것이 아니라, 에이전트를 프롬프트하는 루프를 설계해야 한다고 썼고 큰 반응을 얻었다. 거의 같은 시점에 Anthropic의 Claude Code 리드 Boris Cherny도 자신은 더 이상 Claude를 직접 프롬프트하지 않고, Claude를 프롬프트하며 무엇을 해야 할지 찾아내는 루프를 돌린다고 말했다. Osmani는 6월 7일 이를 “Loop Engineering”이라는 제목으로 정리하고, 다음 날 Substack에도 게시했다. 하나의 점화, 하나의 반향, 하나의 이름이 모두 한 주 안에 나온 셈이다.

I-C 왜 지금 이 용어가 도착했는가

세 사람이 따로따로 같은 말을 꺼낸 이유는 주변 tool들이 조용히 임계점을 넘었기 때문이다. Coding agent는 방치해도 의미 있는 작업을 끝낼 만큼 안정화되었고, 주요 harness에는 scheduling primitive가 생겼으며, 에이전트 한 번을 돌리는 비용도 반복 run이 낭비처럼 보이지 않을 만큼 낮아졌다. 모든 부품이 준비되면 그것들을 결합하려는 움직임은 여러 사람에게 동시에 자명해진다. 이름은 실천보다 늦게 도착했다.

I-D Harness 위 한 층

예전에는 사람이 에이전트에게 한 줄씩 prompt했고, 에이전트는 하나를 끝내면 멈춰서 기다렸다. 사람은 loop 안의 시계였다. 새 방식에서는 스스로 tick을 내는 구조를 설계한다. 이 구조는 timer에 맞춰 실행되고, subagent를 띄워 일을 맡기며, 결과를 다시 자신에게 공급한다. Harness가 단일 agent run을 감싸는 층이라면, loop는 그 run이 스스로 반복되게 만드는 한 층 위의 구조다.

II. Prompt에서 Context를 거쳐 Loop로

“XX Engineering”들은 서로를 대체하지 않는다. 각각 더 큰 단위를 다루며 위로 쌓인다. 한 층 올라갈수록 관심 단위는

표 I
네 층 stack

Layer	Scope	Design question
Prompt Engineering	prompt	모델에게 무엇을 말할까
Context Engineering	context	무엇을 넣고, 무엇을 뺄까
Harness Engineering	tool, action, done criteria	한 번의 run을 어떻게 통제할까
Loop Engineering	schedule, state, feedback	다음 run은 언제, 어떤 state로 시작할까

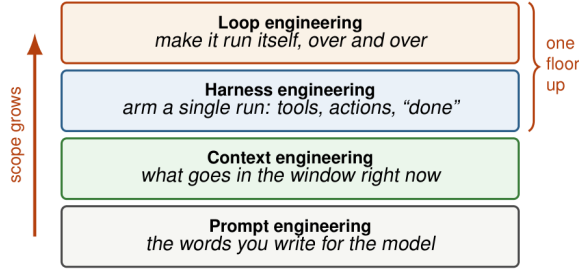


그림 1. 네 층 stack. 각 층은 아래층보다 더 큰 범위를 맡는다. Loop Engineering은 harness가 남겨 둔 “waiting for you” 상태를 자동화한다.

한 문장, 한 context window, 한 번의 run, 그리고 스스로 반복되는 loop로 커진다.

II-A 네 층

Prompt Engineering은 하나의 입력 문장을 잘 쓰는 일이다. Context Engineering은 지금 context window에 무엇을 넣을지, 무엇을 요약하거나 버릴지 결정한다. Harness Engineering은 한 번의 run을 tool, permission, stop condition으로 감싼다. Loop Engineering은 그 run 위에 schedule과 state, verification, 다음 run을 엮는다. Loop는 단순히 한 번 더 실행하는 것이 아니라, 실행이 끝난 뒤 무엇을 저장하고, 무엇을 다시 공급하며, 언제 다시 시작할지를 설계한다.

II-B 한 층 위가 실제로 더하는 것

Harness와 loop를 가르는 동사는 세 가지다. 첫째, timer에 맞춰 실행된다. 둘째, 일을 잘라 subagent를 만든다. 셋째, 결과를 다음 turn으로 되먹인다. 에이전트가 한 번은 깔끔하게 실행되더라도, harness만으로는 다시 깨우고 다음 일을 찾는 부분이 남는다. Loop는 바로 그 “대기”를 제거한다.

II-C 각 층의 실패 반경

같은 오해도 어느 층에 있느냐에 따라 피해가 달라진다. Prompt 층의 오해는 한 답변을 망친다. Context 층의 오해는 한 session을 흐린다. Harness 층의 오해는 한 번의 파일 변경이나 run으로 끝날 수 있다. 하지만 loop 층의 오해는 state file에 기록되고, 다음 날 다시 읽혀 전제로 굳어질 수

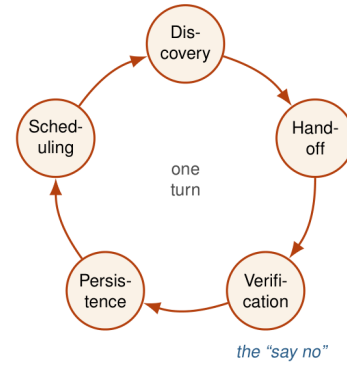


그림 2. 한 turn의 다섯 move. Scheduling은 cycle을 닫아 끝나지 않은 turn을 다음 날 run으로 넘긴다. Verification은 “아니오”라고 말할 수 있는 move다.

표 II
다섯 move와 triage loop의 대응

Move	하는 일	triage loop에서
Discovery	이번 turn의 일을 스스로 찾는다	Skill이 CI, issue, commit을 읽는다
Handoff	작업을 격리해 넘긴다	각 항목이 worktree를 연다
Verification	다른 에이전트가 “아니오”를 말한다	두 번째 subagent가 test와 비교한다
Persistence	대화 밖에 state를 쓴다	PR, inbox, state file
Scheduling	turn을 반복되게 만든다	아침 automation이 스스로 실행된다

있다. Loop는 실수를 여러 turn 동안 살아남게 하는 기계가 될 수 있으므로, 이후의 evaluator와 budget limit, human checkpoint는 모두 실수와 발견 사이의 거리를 줄이기 위해 존재한다.

III. 한 loop의 다섯 move

“Loop”는 빈 회전이 아니다. 한 turn은 실제 작업을 수행한다. 할 일을 찾고, 에이전트에게 넘기고, 결과가 맞는지 검증하고, state를 저장하고, 다음 단계를 결정한다. 이 다섯 가지 중 하나라도 빠지면 loop는 돌지 않거나 제자리에서 돈다.

III-A 구체적인 예

Osmani의 아침 triage loop는 자동으로 시작된다. Triage skill은 전날 실패한 CI, 열려 있는 issue, 최근 commit을 읽고 결과를 markdown file이나 Linear board에 쓴다. 처리할 가치가 있는 항목마다 격리된 worktree를 열고, 한 subagent가 수정안을 만들며, 다른 subagent가 project skill과 test에 비추어 검토한다. Connector는 PR을 열고 ticket을 갱신한다. 감당할 수 없는 것은 사람의 inbox로 간다. State file이 남기 때문에 다음 날은 전날의 연속으로 시작된다.

Verification은 가장 줄이기 쉬우면서도, 줄여서는 안 되

표 III
여섯 부품과 다섯 동작

부품	의미	대응 동작
Automation	schedule 또는 trigger로 실행	Scheduling
Worktree	병렬 에이전트용 격리 디렉터리	Handoff
Skill	영구 지식, 의도 부채 감소	Discovery
Connector	외부 system 연결	Persistence/Discovery
Subagent	generator와 evaluator 분리	Verification
Memory	디스크의 persistent state	Persistence

는 부분이다. Generator가 만든 코드를 같은 generator가 평가하면 자기 숙제 채점이 된다. Persistence는 대화 창이 사라져도 남는 기억이고, scheduling은 한 번의 run을 loop로 바꾸는 장치다.

IV. 여섯 부품

다섯 move는 보통 여섯 부품으로 구현된다. Automation은 trigger와 schedule을 제공한다. Worktree는 병렬 에이전트가 서로의 파일을 밟지 않도록 격리한다. Skill은 context window에 붙여 넣던 거대한 지시문을 업데이트 가능한 영구 지식으로 바꾼다. Connector는 issue tracker, database, staging API, Slack 같은 외부 세계와 loop를 잇는다. Subagent는 작성자와 심판을 분리한다. Memory는 대화 밖의 persistent state다.

V. Generator와 evaluator

Loop에서 가장 어려운 일은 에이전트를 실행시키는 것이 아니라, 내부에 “아니오”라고 말할 수 있는 장치를 넣는 것이다. 코드를 쓴 에이전트는 그 코드를 거절할 가능성이 가장 낮다.

V-A 자기 자신을 늘 칭찬한다

에이전트에게 방금 만든 산출물을 채점하라고 하면, 사람이 보기에는 평범하거나 틀린 결과도 자신 있게 칭찬하는 경향이 있다. 이는 지능의 문제가 아니라 자기 숙제 채점의 문제다. 코드를 쓴 context에는 이미 “왜 이렇게 썼는지”에 대한 자기 설득이 가득 들어 있으므로, 같은 에이전트는 결과 자체보다 그 설득의 궤적을 보게 된다.

V-B 겸손한 작성자를 고치기보다 회의적인 심판을 조정하라

Generator를 더 자기비판적으로 만드는 일은 잘 되지 않는다. 독립 evaluator를 회의적으로 조정하는 편이 훨씬 더 쉽다. 작성자에게 자신의 관점 밖으로 완전히 나가라고 요구할 수는 없지만, 다른 지시와 때로는 다른 모델을 가진 evaluator를 투입할 수는 있다. 이는 하나가 만들고 다른 하나가 결함을 찾는 GAN의 구도를 code generation과 review로 옮긴 것과 닮았다.

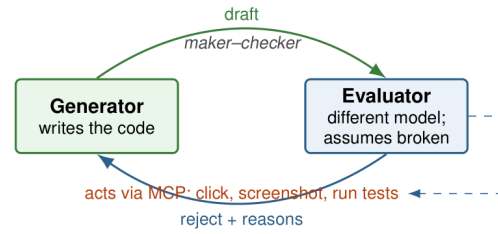


그림 3. Generator와 evaluator를 별도 에이전트로 둔 구조. Evaluator는 generator의 자기 설득을 공유하지 않고, 기본값을 의심으로 두며, 단순히 읽는 대신 행동으로 검증한다.

V-C Evaluator는 읽기만 해서는 안 되고 행동해야 한다

Evaluator가 코드만 읽으면 “그럴듯한가”를 판단할 뿐 “실제로 동작하는가”를 판단하지 못한다. Frontend 작업에서는 evaluator를 Playwright MCP에 연결해 페이지를 열고, 버튼을 누르고, screenshot을 찍고, DOM을 검사하게 할 수 있다. 판단의 기준이 “JSX가 맞아 보인다”에서 “버튼을 눌렀고, 페이지가 이동했으며, 확인 가능한 screenshot이 있다”로 바뀐다.

V-D 제품 안에서: 종료 조건의 /goal

Claude Code의 /goal은 이 구조를 primitive로 만든다. 에이전트에게 조건을 주고, 조건이 충족될 때까지 돌게 한다. 대표적인 evaluator 설정은 다음과 같다.

```

# Evaluator agent (.claude/agents/reviewer.md)
ROLE: 적대적 코드 리뷰어.
ASSUME: 이 코드는 증명되기 전까지 망가져 있다.
DO NOT praise. 실패 지점을 찾아라.
CHECK:
1. 실행되는가?
2. 테스트를 돌리고 실제 출력을 붙였는가?
3. 작성자가 놓친 edge case는?
4. 동작이 티켓과 맞는가?
USE Playwright MCP: 페이지를 열고 클릭하고
스크린샷을 찍고 DOM을 검사하라.
VERDICT: 모든 검사를 통과할 때만 PASS.
아니면 REJECT와 이유 목록을 반환.

```

```

/goal all tests in test/auth pass
and the lint step is clean

```

완료 여부는 작업한 에이전트가 아니라 신선한 작은 모델이 판단한다. 이것이 maker-checker 원리다. Loop의 바닥은 evaluator다. Generator의 수준은 loop가 무엇을 만들 수 있는지를 정하고, evaluator의 수준은 loop가 무엇을 만들지 않을지를 정한다.

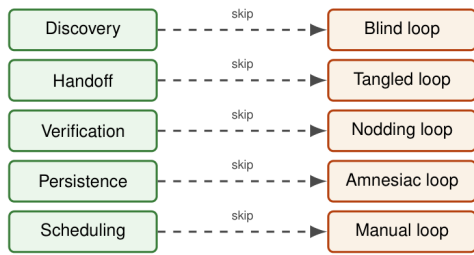


그림 4. 각 안티패턴은 하나의 동작이 빠진 경우다. 다섯 실패는 한 턴의 다섯 동작에 일대일로 대응한다.

VI. 루프가 잘못되는 다섯 방식

실패는 성공보다 더 자주, 더 많은 것을 가르친다. 아래의 다섯 안티패턴은 각각 다섯 동작 중 하나가 빠진 경우다.

VI-A 꼬덕이는 루프: 검증 누락

가장 흔한 실패다. 루프는 돌고 에이전트는 코드를 쓰며, 같은 에이전트가 이를 좋다고 선언한다. 독립 검사가 없으면 패턴은 자기 승인 산출물을 만들고, 루프는 그럴듯한 실수를 기계 속도로 축적한다.

VI-B 기억상실 루프: 지속성 누락

루프가 좋은 일을 찾아 수행했지만 결과가 대화창 안에만 남아 곧 사라진다. 다음 턴은 같은 일을 다시 발견하거나, 더 나쁘게는 같은 일을 다시 수행해 충돌을 만든다. 해결책은 디스크의 상태 파일이다. 에이전트는 잊지만 저장소는 잊지 않는다.

VI-C 수동 루프: 스케줄링 누락

네 동작이 좋아도 자동화가 없으면 루프가 아니라 사람이 기억해서 실행해야 하는 스크립트다. 데모한 날에는 훌륭하지만, 관심이 흩어지는 순간 조용히 멈춘다.

VI-D 눈먼 루프: 발견 누락

사람이 매일 “이 세 버그를 고쳐라”라고 일을 건넌다면 수행은 자동화했지만 발견은 자동화하지 못한 것이다. 무엇을 할지 고르는 일이 실제로 비싼 경우가 많으므로 절감 효과가 줄어든다.

VI-E 얽힌 루프: 인계 누락

여러 에이전트가 같은 디렉터리를 공유하면 병렬성이 곧 문제로 드러난다. 단일 에이전트 루프는 괜찮아 보일 수 있지만, 다섯 에이전트가 한꺼번에 돌기 시작한 아침에는 문제가 나타난다. 해결책은 작업마다 격리된 worktree를 쓰는 것이다.

VII. 실제로 도는 루프

실제로 돌아가는 루프는 모델이 강해서가 아니라 골격이 단단해서 돈다. 트리거가 시작을 누르고, 제약이 레일을 만들며, 끝에는 인간 체크포인트가 있다.

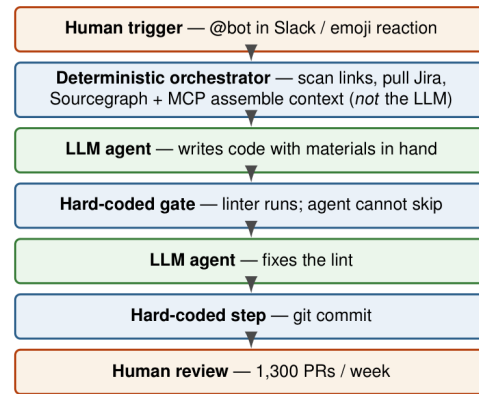


그림 5. Stripe의 Minions 파이프라인. 결정론적 게이트(파란색)와 LLM 단계(초록색)가 맞물린다. 규칙으로 묶을 수 있는 것은 확률 모델 밖에 둔다. 신뢰성은 모델 크기가 아니라 제약에서 나온다.

VII-A 한 엔지니어의 아침

Osmani의 트리아지 루프는 매일 아침 자동 실행된다. 중요한 점은 자동화가 거대한 지시문을 직접 품고 있는 것이 아니라 스킬을 호출한다는 것이다. 스킬은 시간이 지나며 수정되고 개선될 수 있다. 개인이 루프 하나를 돌리는 모습은 이렇다. 한 사람, 한 기계, 매일 아침 반복되는 허드렛일이 자동으로 처리된다.

VII-B Stripe의 Minions: 주당 1,300개 PR

기업 규모의 사례는 Stripe의 Minions다. How I AI 팟캐스트에서 Stripe 엔지니어 Steve Kaliski가 설명한 바에 따르면, 주당 1,300개가 넘는 pull request가 사람이 한 줄도 직접 쓰지 않은 채 병합된다. 트리거는 가볍다. Slack에서 봇을 부르거나 emoji reaction을 달면 된다. 신뢰성을 만드는 것은 모델이 깨어나기 전의 결정론적 오케스트레이터다. 링크를 스캔하고 Jira를 가져오고, 문서를 찾고, Sourcegraph와 MCP로 관련 코드를 찾는다. 규칙으로 풀 수 있는 것은 확률 모델에게 맡기지 않는다.

Minions의 핵심 주장은 더 강한 모델이 아니라 더 좋은 제약이다. 이 아키텍처는 결정론적 게이트와 창의적 LLM 단계를 엮는다. 에이전트가 코드를 쓰면, 하드코딩된 파이프라인이 linter를 돌리며 에이전트는 이를 건너뛸 수 없다. 에이전트가 lint를 고치고 나면, 하드코딩된 단계가 커밋을 만든다. 1,300개의 PR도 여전히 사람이 리뷰한다. 사람은 사라진 것이 아니라 작성자 자리에서 검토자 자리로 옮겨갔다.

VII-C “자는 동안”이 실제로 의존하는 것

로컬 /loop와 데스크톱 스케줄 작업은 기계가 켜져 있어야 한다. 기계를 끄면 멈춘다. 로컬 상태를 볼 필요가 있으면 로컬 루프가 맞고, 로컬 상태와 분리해 기계가 꺼져 있어도 돌게 하려면 Cloud Routines나 GitHub Actions 스케줄이

표 IV
스케줄링 옵션 비교

항목	Cloud	Desktop	/loop
실행 위치	클라우드	로컬 기계	로컬 기계
기계 차지 필요	아니오	예	예
세션 열림 필요	아니오	아니오	예
최소 간격	1시간	1분	1분
로컬 파일 접근	아니오	예	예

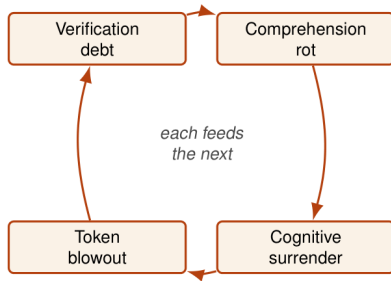


그림 6. 네 비용은 서로를 강화한다. 검증되지 않은 산출물은 이해를 침식하고, 이해의 침식은 항목을 부르며, 항목은 루프를 더 오래 방치하게 만들고, 그 결과 더 많은 비용과 더 많은 미검증 산출물이 생긴다.

맛다. 어떤 스케줄러도 모든 조건을 동시에 만족하지 않는다.

VIII. 비용: 저절로 닫히지 않는 네 텀

스스로 도는 루프는 스스로 실수할 수 있는 루프이기도 하다. 루프가 기쁘게 달릴수록 오류는 더 조용히 쌓인다.

검증 부채는 병합된 PR이 시간을 절약하는 동시에 검증되지 않은 산출물을 빗으로 남기는 현상이다. 이해 부식은 사람이 코드를 읽지 않고 병합하면서 코드베이스에 대한 머릿속 모델이 뒤쳐지는 현상이다. 인지적 항목은 루프가 너무 매끄럽게 돌아 사람이 더 이상 읽지 않게 되는 상태다. 토큰 폭증은 루프가 하위 에이전트를 낳고 재시도하며 밤새 비용을 태우는 문제다.

예를 들어 루프가 밤새 20개의 PR을 열었고 모두 테스트가 초록색이라고 하자. 그중 세 개가 테스트가 잡지 못한 미묘한 오류를 담고 있다면, 독립 평가기가 없는 순간 그것은 병합된다. 이것이 검증 부채다. 사람이 20개를 읽지 않고 병합했으므로 코드베이스에 대한 이해는 20개 변경만큼 뒤쳐진다. 루프가 너무 잘 돌아간다는 이유로 다음 날 배치를 아예 읽지 않는다면 인지적 항목이다. 밤새 재시도와 하위 에이전트가 과하게 돌아간다면 비용도 예상보다 커진다. 네 비용은 하나의 실패가 드러나는 네 가지 얼굴이다.

IX. Go 버튼만 누르는 사람이 아니라 엔지니어로 남기

같은 루프도 두 사람이 만들면 정반대의 결과를 낼 수 있다. 한 사람은 이미 이해하고 있는 일에서 더 빨라지기 위해 루프를 사용한다. 코드를 읽고 방향 감각을 유지하며, 루프는 이미 가진 판단을 확장한다. 다른 사람은 더 이상 이해하지

표 V
첫 루프 체크리스트

요소	스스로 물을 질문
발견	루프가 오늘의 일을 스스로 찾는가
인계	각 작업이 격리되어 있는가
검증	작성자과 다른 주체가 실제로 실행해 보는가
지속성	대화가 사라져도 상태가 남는가
스케줄링	사람이 기억하지 않아도 다시 도는가
인간 체크포인트	병합·배포·큰 비용 전에 사람이 멈출 수 있는가

않기 위해 같은 루프를 사용한다. 여섯 달 뒤 한 사람은 더 강해지고, 다른 사람은 읽을 수 없는 기계의 문지기가 된다.

루프는 고정된 품질을 가진 도구가 아니다. 루프는 가져온 것을 그대로 증폭한다. 이해를 가져오면 이해를 증폭하고, 게으름을 가져오면 게으름을 증폭한다. 생성은 극도로 싸진다. 코드, 계획, PR, 수정안은 거의 공짜가 된다. 희소한 것은 판단이다. 어떤 계획이 맞는지, 어느 지점에서 멈춰야 하는지, 실행은 되지만 뿌리에서 틀린 산출물이 무엇인지 아는 능력이다.

X. 첫 루프를 만드는 방법

첫 루프는 작고 지루해야 한다. 매일 반복되고, 입력이 기계적으로 발견 가능하며, 실패해도 재앙이 아닌 일을 고른다. 예를 들어 “매일 아침 실패한 CI를 읽고, 작고 명백한 실패에 대해 격리된 worktree에서 수정안을 만들고, 테스트를 돌린 뒤 PR 초안과 사람에게 보낼 요약의 남긴다” 정도가 좋다.

실행 절차는 단순하다. 먼저 입력을 정한다. 그 입력을 읽는 스킬을 만든다. 작업마다 worktree를 연다. 생성기에게 수정하게 한다. 별도 평가기가 테스트와 동작을 확인한다. 결과를 PR, 받은함, 상태 파일에 남긴다. 마지막으로 타이머를 붙인다. 그리고 첫 주에는 반드시 로그와 비용, 실패 이유를 매일 읽는다. 루프는 만든 뒤 방치하는 장치가 아니라, 판단을 어디에 둘지 새로 설계하는 장치다.

XI. 이 글에서 쓰는 용어

XII. 결론

Loop Engineering은 에이전트를 더 잘 다루는 새로운 요령이 아니라, 사람이 에이전트와 관계 맺는 위치를 바꾸는 설계다. 사람은 prompt를 한 줄씩 쓰는 operator에서, prompt를 보내는 system을 설계하는 engineer가 된다. 그 변화는 강력하지만 위험하다. Generation이 싸지는 만큼 judgment는 더 중요해지고, automation이 쉬워지는 만큼 “아니오”라고 말하는 장치를 loop 안에 넣어야 한다. 좋은 loop는 사람을 없애지 않는다. 좋은 loop는 사람이 있어야 할 자리를 더 분명하게 만든다.

표 VI
용어

용어	의미
Prompt Engineering	모델에게 주는 단일 입력을 설계하는 일
Context Engineering	context window에 넣을 정보와 뺄 정보를 설계하는 일
Harness Engineering	단일 run의 tool, permission, stop condition을 설계하는 일
Loop Engineering	harness가 반복 실행되도록 schedule, state, verification을 설계하는 일
Generator	산출물을 만드는 에이전트
Evaluator	산출물을 독립적으로 검증하고 거절할 수 있는 에이전트
Persistence	대화 밖에 남는 state

출처와 재구성 메모

이 한국어 PDF는 공개 공유된 *Loop Engineering: The Anthropic Playbook for Designing Systems That Prompt Your Agents* PDF에서 추출한 텍스트와 원본 PDF에서 크롭한 그림을 바탕으로 만든 재구성·번역본이다. 원본 TeX 소스는 공개적으로 확인되지 않았으며, 본 파일은 원저자의 원본 TeX가 아니다.